

WELCOME TO THE TEAM

Alex Tan · Backend Engineer. 2026-10-05 to
2027-01-03. Read this first, then open the context
folder in Bob.

OCT 05

CONTENTS

01	WHY WE EXIST	P. 3
02	WHAT YOU OWN	P. 3
03	TIMELINE	P. 3
04	ARCHITECTURE AT A GLANCE	P. 4
05	RUN IT FIRST	P. 4
06	CODE TOUR	P. 4
07	ISSUES WE FOUND	P. 5
08	STARTER TASKS	P. 5
09	PEOPLE TO MEET	P. 6
10	WHERE THINGS LIVE	P. 6
11	DAY-1 ACCOUNTS & ACCESS	P. 6
12	WEEK-1 CHECKPOINTS	P. 6
13	SOURCES	P. 7

YOU

ALEX TAN

Backend Engineer. Level: junior. Work email:
alex.tan@example.com

Northwind Labs

DAY

TIMELINE

2026-10-05 to 2027-01-03, 90 days.

Start 2026-10-05

End 2027-01-03

YOUR PEOPLE

- Manager: Priya Menon
- Buddy: Daniel Wong

START HERE

- Read this pack, then watch the video if it came with your email
- Set up Bob IDE with the context zip: steps in your email and in context/README.md
- Then follow the Timeline section

WELCOME TO NORTHWIND LABS, ALEX

Role: Backend Engineer · Level: Junior · Manager: Priya Menon · Buddy: Daniel Wong · Start: 2026-10-05 · End: 2027-01-03

SECTION 01

WHY WE EXIST

Northwind Labs builds field-service logistics software — tools simple enough for a two-person team to run from a phone. The guiding values are: **ship small, ship often; quality is everyone's job; write it down; customer first, not customer only; and own it end to end.**

The project you are joining is the **Cinema Ticket Management System (CTMS)**: a Jakarta EE web application that manages movies, showtimes, seat bookings, and payments for a cinema chain. It is built in Java with JSP views, backed by PostgreSQL (hosted on Supabase), and uses the Stripe API for payments.

Source: [company/handbook.md](#)

SECTION 02

WHAT YOU OWN

You are on the **Platform team**, which owns the API and data layer. In the CTMS codebase that means:

- **com.ctms.controller** — Servlet classes that receive HTTP requests and direct them to the right service. 18 servlets registered in **web/WEB-INF/web.xml**.
- **com.ctms.service** — Business logic: AccountService, MovieService, ShowtimeService, PaymentService, MenuService.
- **com.ctms.dao** — All SQL runs here via JDBC PreparedStatements: CustomerDAO, EmployeeDAO, MovieDAO, ShowtimeDAO, PaymentDAO, FoodDAO.
- **com.ctms.model** — Plain Java objects representing DB rows: Customer, Employee, Movie, ShowTime, Payment, Food.
- **com.ctms.filter** — AuthFilter.java guards protected routes by checking **userType** in the session.
- **com.ctms.util** — ConnectionDB.java opens a JDBC connection to Supabase PostgreSQL.

Source: [web/WEB-INF/web.xml](#), [src/java/com/ctms/](#) package scan

SECTION 03

TIMELINE

PHASE	START	END	GOAL
1	2026-10-05	2026-10-11	Setup and comprehension
2	2026-10-12	2026-11-03	First contributions
3	2026-11-04	2026-12-03	Ownership
4	2026-12-04	2027-01-03	Independence

SECTION 04

ARCHITECTURE AT A GLANCE

CTMS is a classic three-tier Jakarta EE web application:

Browser → Tomcat Servlet container → PostgreSQL on Supabase
↑ HTTPS
Stripe API (payments)

Context path: `/SAD_Project` (source: `web/META-INF/context.xml`)

Access control: Two filters declared in `web/WEB-INF/web.xml` — `EmployeeAuthFilter` protects `/employee/*`;

`CustomerAuthFilter` protects `/customer/profile.jsp`. Both are implemented by `src/java/com/ctms/filter/AuthFilter.java`.

Employee permission levels (from `src/java/com/ctms/controller/LoginServlet.java:63-72`):

RESTRICTION LEVEL	ROLE	POST-LOGIN REDIRECT
3	HQ (head office)	<code>/HomeServlet</code>
2	Manager	<code>/ShowtimeServlet</code>
1	Staff	<code>/HomeServlet</code>

Database connection: Every DAO opens a new `ConnectionDB` in its constructor (e.g. `src/java/com/ctms/dao/CustomerDAO.java:14-16`). There is no connection pool — this is ISSUE-006.

Payment flow: `PaymentServlet` creates a Stripe `PaymentIntent` via `PaymentService` (`src/java/com/ctms/service/PaymentService.java:14-16`), stores the result via `PaymentDAO`, and forwards the `clientSecret` to `checkout.jsp`. Stripe calls back to `WebhookServlet` (`src/java/com/ctms/controller/WebhookServlet.java:31-71`) to mark payments succeeded or failed.

SECTION 05

RUN IT FIRST

1. Open the project in **NetBeans** (it uses `nbproject/` configuration files).
2. Build: `ant clean && ant` (produces a WAR in `dist/`).
3. Deploy the WAR (renamed `SAD_Project.war`) to your Tomcat `webapps/` directory.
4. Open `http://localhost:8080/SAD_Project/` in a browser.

No git remote is configured. Ask Daniel Wong how source control works for this project. JDK version required: Unknown — ask your buddy.

SECTION 06

CODE TOUR

STOP	FILE	WHAT TO LOOK AT
1	<code>src/java/com/ctms/util/ConnectionDB.java:6-9</code>	JDBC URL and username fields — note credentials are hardcoded (ISSUE-001)
2	<code>src/java/com/ctms/filter/AuthFilter.java:18-55</code>	How session-based auth guards routes by role
3	<code>src/java/com/ctms/controller/LoginServlet.java:19-101</code>	Full login flow: read params → verify → set session → redirect by restriction level

STOP	FILE	WHAT TO LOOK AT
4	<code>src/java/com/ctms/service/AccountService.java:32-44</code>	verifyByEmail : plain-text password comparison (ISSUE-003)
5	<code>src/java/com/ctms/dao/CustomerDAO.java:22-55</code>	findCustomerByEmail : PreparedStatement pattern; also see SQL bug at line 105 (ISSUE-004)
6	<code>src/java/com/ctms/dao/MovieDAO.java:17-36</code>	getAllMovies : how a DAO returns a typed list from a ResultSet
7	<code>src/java/com/ctms/controller/PaymentServlet.java:16-50</code>	Stripe payment intent creation and forward to checkout.jsp
8	<code>src/java/com/ctms/service/PaymentService.java:11-50</code>	Stripe SDK usage; hardcoded API key at line 19 (ISSUE-002)
9	<code>src/java/com/ctms/controller/WebhookServlet.java:17-74</code>	Stripe webhook handler; hardcoded endpoint secret at line 19 (ISSUE-002)

SECTION 07

ISSUES WE FOUND

ID	TITLE	SEVERITY	SUGGESTED TIMELINE
ISSUE-001	Hardcoded Database Credentials in Source Code	critical	2026-10-12 → 2026-10-19
ISSUE-002	Hardcoded Stripe API Key and Webhook Secret	critical	2026-10-12 → 2026-10-19
ISSUE-003	Passwords Stored and Compared in Plain Text	critical	2026-10-19 → 2026-11-03
ISSUE-004	SQL Syntax Bug in CustomerDAO.deleteCustomer	high	2026-10-05 → 2026-10-11
ISSUE-005	No Unit or Integration Tests in the Repository	high	2026-10-19 → 2026-11-03
ISSUE-006	Database Connection Per-Request Leak (No Connection Pool)	high	2026-10-19 → 2026-11-03
ISSUE-007	Duplicate Servlet Declarations in web.xml	medium	2026-10-12 → 2026-10-19
ISSUE-008	AuthFilter Does Not Handle Null Session Attribute Safely	medium	2026-10-19 → 2026-11-03
ISSUE-009	Customer Password Exposed in Session / Model Object	medium	2026-10-19 → 2026-11-03
ISSUE-010	No Input Validation or Output Encoding on JSP Pages	medium	2026-11-04 → 2026-11-17

Full detail (root cause, guidance, verification) is in [context/issues/](#).

SECTION 08

STARTER TASKS

- Quick win (Phase 1, by 2026-10-11)**: Fix the SQL syntax bug in `src/java/com/ctms/dao/CustomerDAO.java:105` — change **"DELETE FROM TABLE customer"** to **"DELETE FROM customer"**. Write a test to verify `deleteCustomer()` returns true for a valid ID.
- Collaborative (Phase 2, by 2026-10-26)**: Remove duplicate servlet declarations from `web/WEB-INF/web.xml:133-147` (ISSUE-007). Pair with Daniel Wong to deploy and smoke-test on Tomcat.
- Real slice (Phase 2, by 2026-11-03)**: Move the database credentials in `src/java/com/ctms/util/ConnectionDB.java:8-15` to environment variables (ISSUE-001). This touches the utility layer and requires coordinating with Priya Menon to rotate the credentials.

SECTION 09

PEOPLE TO MEET

NAME	ROLE	ASK THEM FOR	HOW TO REACH
Priya Menon	Manager	Expectations, access, credential rotation plan	Not specified — ask your manager
Daniel Wong	Buddy	Codebase tour, Git setup, Tomcat deployment, on-call intro	Not specified — ask your manager
IT Help	IT	GitHub invite, Bob IDE seat, VPN/SSO	it-help@northwindlabs.example

SECTION 10

WHERE THINGS LIVE

ITEM	LOCATION
Source code	CTMS_Project/src/java/com/ctms/
JSP views	CTMS_Project/web/
Servlet config	CTMS_Project/web/WEB-INF/web.xml
DB connection	CTMS_Project/src/java/com/ctms/util/ConnectionDB.java
Build config	CTMS_Project/build.xml, CTMS_Project/nbproject/
Libraries	CTMS_Project/web/WEB-INF/lib/
GitHub org	github.com/northwind-labs (ask buddy for repo link)
Team chat	northwindlabs.slack.com — start in #eng-onboarding

SECTION 11

DAY-1 ACCOUNTS & ACCESS

Provisioned by IT and your manager before you start. Raise any missing access with it-help@northwindlabs.example.

- ☐ Laptop and local development environment
- ☐ GitHub organisation invite (github.com/northwind-labs)
- ☐ Slack workspace (northwindlabs.slack.com)
- ☐ Bob IDE seat (bob.ibm.com)
- ☐ VPN/SSO credentials

SECTION 12

WEEK-1 CHECKPOINTS

Answers and dates live in **context/timeline.md**.

1. Is my local environment running without help?
2. Have I read through the three critical security issues (ISSUE-001, ISSUE-002, ISSUE-003)?
3. Have I opened my first pull request?
4. Have I had my Day-5 check-in with Priya Menon?

5. Can I explain the request flow from browser to database and back in my own words?

SECTION 13

SOURCES



- **company/handbook.md** — Northwind Labs company handbook
- **web/WEB-INF/web.xml** — Servlet and filter declarations
- **web/META-INF/context.xml** — Deployment context path
- **src/java/com/ctms/util/ConnectionDB.java** — Database connection
- **src/java/com/ctms/controller/LoginServlet.java** — Login flow and permission levels
- **src/java/com/ctms/service/AccountService.java** — Account verification
- **src/java/com/ctms/dao/CustomerDAO.java** — Customer data access
- **src/java/com/ctms/dao/MovieDAO.java** — Movie data access
- **src/java/com/ctms/filter/AuthFilter.java** — Route protection
- **src/java/com/ctms/controller/PaymentServlet.java** — Payment initiation
- **src/java/com/ctms/service/PaymentService.java** — Stripe integration
- **src/java/com/ctms/controller/WebhookServlet.java** — Stripe webhook
- **roles/backend-engineer.md** — Role definition